

# “榨”出硅的极限：怎么让 GPU 不“闲着”？

——与 SGLang、RadixArk 深聊 AI Infra 技术与千亿美元市场

五道口纳什 & Codex

2026 年 8 月 5 日



视频作者/频道：硅谷 101

发布日期：2026 年 7 月 31 日

视频时长：28 分 47 秒

视频链接：<https://www.bilibili.com/video/BV1FnGA66EPP>

PDF 制作 Skill：<https://github.com/wdkns/wdkns-skills>

# 目录

|                                       |           |
|---------------------------------------|-----------|
| <b>1 引言：从“造芯大战”到 AI Infra 的千亿赛道</b>   | <b>2</b>  |
| <b>2 昂贵的 GPU，大多数时候很“闲”</b>            | <b>2</b>  |
| 2.1 一个反直觉的事实：GPU 在空转                  | 2         |
| 2.2 AI Infra 的四层架构                    | 4         |
| 2.3 本章小结                              | 5         |
| <b>3 优化“计算”：K/V Cache 的极致复用</b>       | <b>5</b>  |
| 3.1 推理中的重复计算问题                        | 5         |
| 3.2 RadixAttention：基数树组织的 KV Cache 共享 | 6         |
| 3.3 工程优雅性：缓存感知调度与淘汰机制                 | 8         |
| 3.4 本章小结                              | 9         |
| <b>4 优化“等待”：PD 分离与连续批处理</b>           | <b>9</b>  |
| 4.1 请求级等待：Continuous Batching         | 9         |
| 4.2 阶段级等待：Prefill/Decode 分离           | 10        |
| 4.3 本章小结                              | 12        |
| <b>5 优化“算力”：低精度计算与投机采样</b>            | <b>12</b> |
| 5.1 GPU 算力为何跑不满                       | 12        |
| 5.2 低精度计算：用更少的 bit 算更多                | 12        |
| 5.3 投机采样（Speculative Decoding）        | 14        |
| 5.4 本章小结                              | 16        |
| <b>6 资源协同：适配 RL 后训练的 Miles</b>        | <b>17</b> |
| 6.1 强化学习对训练-推理协同的新要求                  | 17        |
| 6.2 Miles：SGLang 的训练搭档                | 17        |
| 6.3 本章小结                              | 19        |
| <b>7 更大的野心：AI Infra 成为公共品</b>         | <b>19</b> |
| 7.1 开源推理引擎的商业化浪潮                      | 19        |
| 7.2 “以后没有 Frontier Lab 这个词”           | 21        |
| 7.3 本章小结                              | 22        |
| <b>8 总结与延伸</b>                        | <b>22</b> |
| 8.1 核心脉络回顾                            | 22        |
| 8.2 技术趋势与展望                           | 22        |
| 8.3 开放问题与延伸阅读                         | 23        |

# 1 引言：从“造芯大战”到 AI Infra 的千亿赛道

2026 年, AI 行业突然掀起了一场“造芯大战”。OpenAI 与博通联合研发出首款推理芯片 Jalapeño, 从设计到流片仅用九个月; 谷歌内部代号“Frozen v2”的服务器芯片计划将 Gemini 模型部分架构固化进硅片; Anthropic、智谱、DeepSeek 也相继传出芯片布局。然而, 并非每家公司都有资金与能力重造一块芯片。

与此同时, AI 推理平台 Baseten 一年收入增长 20 倍, 估值从 21 亿美元暴涨至 130 亿; Fireworks 估值七个月内翻四倍达 175 亿美元; 开源推理引擎双子星 vLLM 与 SGLang 相继宣布商业化, 种子轮融资均超 1 亿美元。“GPU 利用率”成为硅谷最新关键词, AI Infra 正在成为千亿美元级的新赛道。

Meta、Google、Microsoft 等科技巨头今年资本开支合计预计超 1 万亿美元, 黄仁勋预测到 2030 年全球 AI 基础设施年投资将达 4 万亿美元。但华尔街的担忧也不无道理——互联网泡沫的前车之鉴仍在, 大量资金投入硬件建设中, 短期可能出现严重问题。那么, 如果不能无限堆卡, 还有什么办法满足 AI 爆发式增长的算力需求? 答案指向了 **AI 基础设施层 (AI Infra) 的系统级优化**。

本视频由硅谷 101 频道制作, 邀请 SGLang 孵化出的 RadixArk 团队核心成员——联合创始人朱邦华 (前英伟达首席研究科学家)、技术团队成员陈震林 Richard、以及前微软能源战略经理 Ethan Xu, 深入探讨 AI Infra 的四层架构与行业如何从系统软件与服务编排层面“榨”出硅的性能极限。

## 2 昂贵的 GPU, 大多数时候很“闲”

### 2.1 一个反直觉的事实: GPU 在空转

人们通常认为, AI 爆发式增长让 GPU 供不应求, 因此 GPU 一定在满负荷运转。但实际情况并非如此。

卡内基梅隆大学 (CMU) 2026 年 4 月发布的一项研究对大型学术 AI 集群中 **756 块 GPU** 进行了 **31 天** 的细粒度遥测, 覆盖从 A6000、L40S、A100、H100 到最新 B200 等六代产品。

表 1: CMU GPU 利用率遥测数据 (756 块 GPU, 31 天)

| 负载类型        | 执行时空转时间占比 | 空转能耗占比 |
|-------------|-----------|--------|
| 整体集群平均      | ~20%      | ~11%   |
| Agentic 类负载 | ——        | 65%    |
| ChatGPT 类请求 | ——        | 52%    |

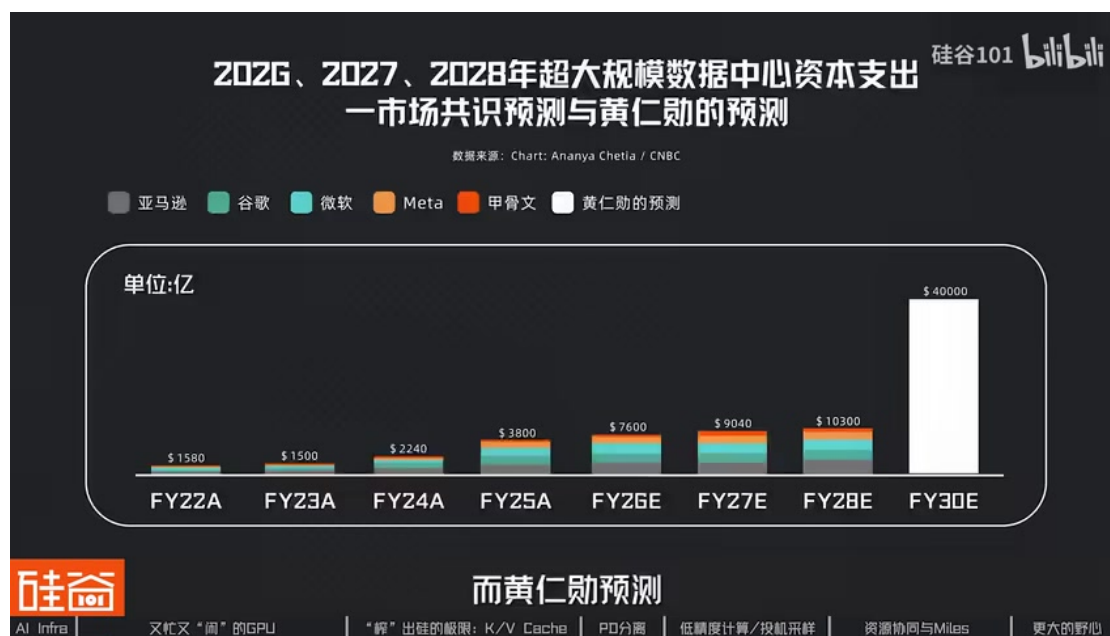


图 1: 视频中的 CMU GPU 利用率遥测数据图<sup>1</sup>

### 核心发现：算力悖论

算力严重短缺与算力大量浪费同时存在。GPU 整体有近 **20% 的执行时间**和约 **11% 的能耗消耗在等待上**。在推理场景中，Agentic 负载的空转能耗高达 **65%**，ChatGPT 类请求也达到 **52%**。

这意味着：在高性能计算集群中，每 5 张 GPU 中约有 1 张的算力被白白浪费。

GPU 只是 AI 系统中的一个环节。从用户请求进入系统到最终返回结果，需要经历：网络传输、资源调度、模型加载、内存管理、GPU 推理等多个步骤。任何一个环节出现瓶颈，GPU 就只能停下来等待。这种“看似繁忙、实则空转”的状态，正是 AI Infra 要解决的核心问题。

2.2 AI Infra 的四层架构

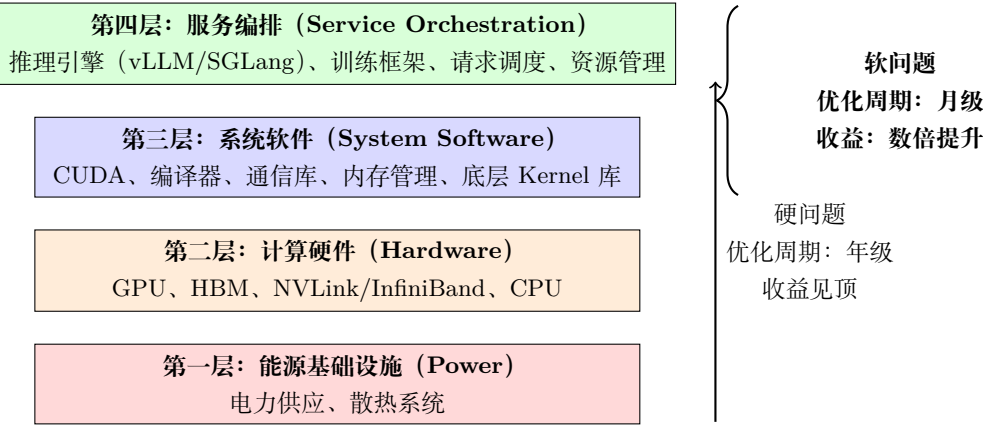


图 2: AI Infra 四层架构：能源、硬件、系统软件、服务编排。上两层为“软”问题，是当前优化的主战场。



图 3: 视频中的 AI Infra 四层架构示意图<sup>2</sup>

优化重心上移

四层架构中，能源基础设施层和计算硬件层是“硬”问题——电力、散热、芯片的改造周期以年为单位，优化空间正在快速见顶。但系统软件层和服务编排层是“软”问题——本质上是工程与算法问题，优化后可带来**数倍的性能提升**。

正如嘉宾 Richard 所指出的：“尽管成本主要发生在物理层，但大部分工作都要放在软件层和服务编排，因为人力投入到这一块，能带来最大的优化可能性。”

举一个具体例子：一台 NVIDIA GB200 NVL72 机柜采购成本约 400 万美元，如果 GPU 实际利用率只有 50%，相当于 200 万美元以电费、折旧和机会成本的形式被浪费。若能把利用率从 50% 推到 90% 以上，效果相当于**凭空多出一台机柜**。这就是软件优化撬动的商业价值。

Anthropic 内部 AI Infra 工程师已占员工最大比例，推理团队超 200 人，三年间从 infra 极度不稳定到极度稳定，直接帮助其在 2026 年二季度实现盈利。这些都说明：**当硬件成本高到如此量级，软件层的每一次优化都是直接的商业问题。**

## 2.3 本章小结

GPU 的“忙碌”大多是一个假象。CMU 的遥测数据揭示了算力利用率的系统性浪费：在整体集群中约 20% 的执行时间和 11% 的能耗被用于等待而非计算。AI Infra 的四层架构中，能源与硬件层的优化空间正在见顶，而系统软件与服务编排层仍有数倍的优化空间，这构成了当前千亿美元 AI Infra 赛道的核心逻辑。

## 3 优化“计算”：K/V Cache 的极致复用

### 3.1 推理中的重复计算问题

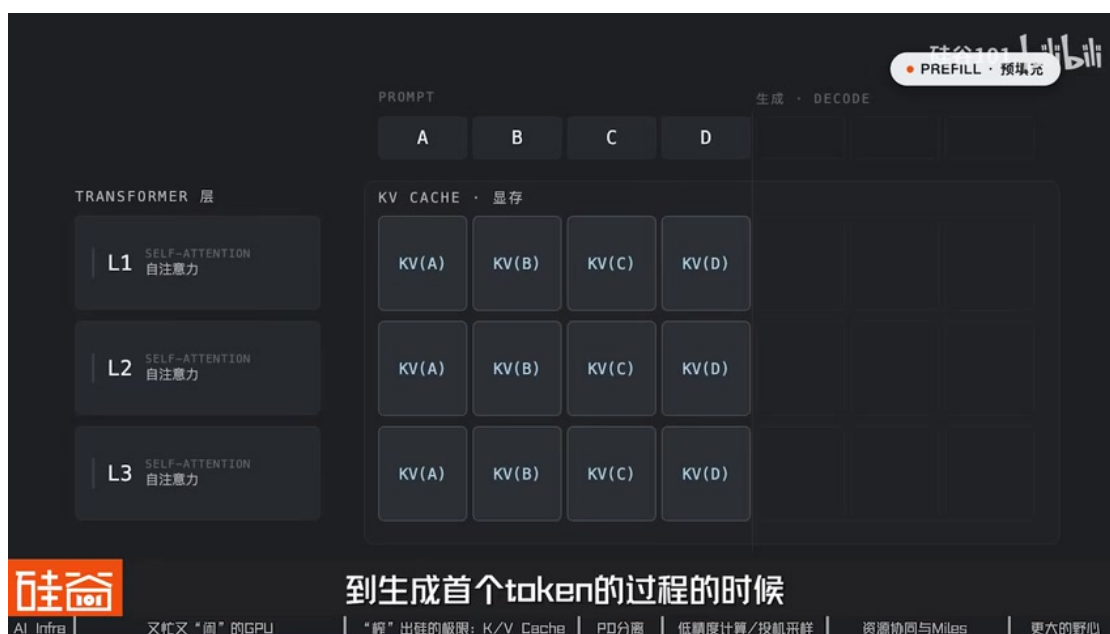


图 4: 视频中 KV Cache 原理可视化讲解<sup>3</sup>

大模型推理有一个被业界称为“推理税”的痛点：当同一段文字被反复喂给模型时，每次都要从头计算一遍。在 Agent 工作流中，同样的工具描述可能被调用几十次，每次都从零开始。Anthropic 曾通过 KV Cache 复用优化将成本直接降低 90%。

<sup>3</sup>视频画面时间区间：00:10:30。

### Transformer 自回归解码与 KV Cache

Transformer 在自回归解码时，每生成一个新 token  $x_t$ ，需要对所有历史 token  $\{x_1, \dots, x_{t-1}\}$  计算 self-attention：

- 对每个历史 token  $i$  计算查询向量  $Q_i$ 、键向量  $K_i$ 、值向量  $V_i$
- Attention 输出为： $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
- **关键观察**：  $K_i$  和  $V_i$  对于已生成的 token 是不变的。因此可以缓存这些向量，避免重复计算

设输入序列长度为  $n$ ，隐藏维度为  $d$ ，层数为  $L$ ，则：

- **无 KV Cache 时**：每次生成新 token 的计算复杂度为  $O(L \cdot n \cdot d^2)$ ，随序列长度平方增长
- **有 KV Cache 时**：每次生成新 token 仅需计算当前 token 的 KV 并追加，复杂度降为  $O(L \cdot d^2)$ ，与序列长度无关

但代价是显存占用：每层需存储  $2 \times n \times d$  个浮点数的 KV 缓存，总 KV Cache 大小约为：

$$M_{KV} = 2 \times L \times n \times d \times \text{bytes\_per\_element}$$

对于典型场景 (Llama-70B 级别,  $L = 80$ ,  $d = 8192$ , FP16)，单个请求在 8K 上下文下的 KV Cache 约需  $2 \times 80 \times 8192 \times 8192 \times 2 \approx 20\text{GB}$ 。

## 3.2 RadixAttention: 基数树组织的 KV Cache 共享

SGLang 的核心创新之一是 **RadixAttention**，它使用**基数树 (Radix Tree)** 数据结构来组织跨请求的 KV Cache。

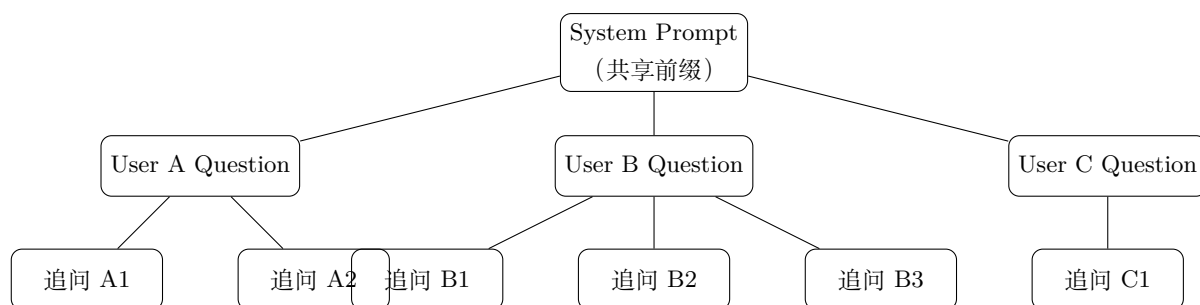


图 5: Radix Tree 组织 KV Cache 的示意：共享同样前缀的请求共用同一段枝干的 KV 缓存，仅在分叉处开始独立计算。

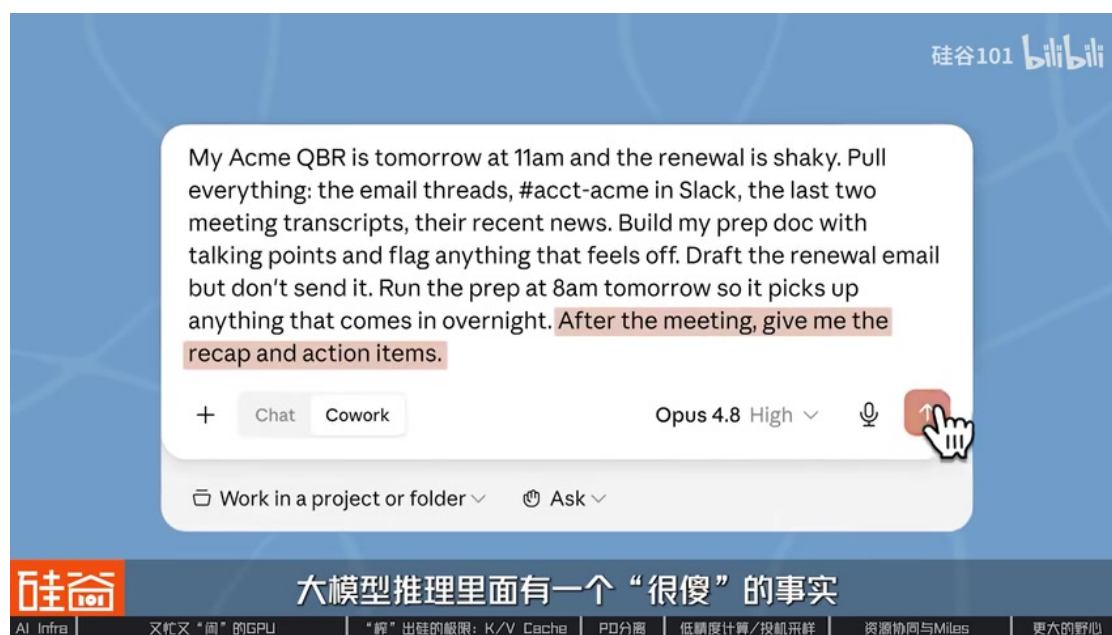


图 6: 视频中的 Radix Tree 基数树结构演示<sup>4</sup>

Radix Tree 将海量请求的 KV Cache 组织成一棵**共享前缀树**。共享同样开头的请求共用同一段枝干的 KV 缓存，只有当内容开始分叉时才各自长出新的树枝。在 Agent 场景下，共享的系统提示词形成树根，不同用户提示词形成一级分支，后续对话形成更深的分支。

### KV Cache 复用的性能增益

传统方案中每个请求独立维护 KV Cache，重复的系统提示词每次都需要重新计算。Radix-Attention 将 KV Cache 变成**全局可复用的资源**：

- **前缀命中率**：在 Agent 工作流中，系统提示词的 KV Cache 命中率可接近 100%
- **缓存共享效率**： $N$  个共享相同系统提示词的请求，KV Cache 存储需求从  $N \times M_{sys}$  降为  $1 \times M_{sys}$
- **延迟与吞吐**：省去了重复的 prefill 计算，首 token 延迟（TTFT）显著降低；释放的算力可用于处理更多并发请求



### 3.3 工程优雅性：缓存感知调度与淘汰机制

仅有好的缓存结构还不够，还需要调度器的配合。SGLang 实现了多项工程优化：



图 7: 视频中的缓存感知调度示意图<sup>5</sup>

- **Cache-aware Scheduling (缓存感知调度)**：将前缀相似的请求排在一起处理。类比餐厅后厨——连续处理三份麻辣香锅可以复用调料和准备工作，而交替处理麻辣香锅、寿司、牛排则会频繁洗锅换料。SGLang 让前缀相似请求靠近处理，最大化缓存命中率。



图 8: 视频中的 Eviction 淘汰机制示意图<sup>6</sup>

<sup>5</sup>视频画面时间区间：00:11:30。

<sup>6</sup>视频画面时间区间：00:13:00。

- **Eviction (淘汰机制)**: 显存不够时, 丢弃复用价值最低的缓存。常用系统提示词、高频 RAG 文档、最近被访问的对话前缀优先保留; 很久未用、复用价值低的 KV Cache 优先淘汰。这本质上是一个近似 LRU (Least Recently Used) 策略, 结合了访问频率与时间局部性。
- **分布式 Cache-aware Load Balancer (缓存感知负载均衡)**: 大规模服务通常由多 GPU 组成。如果某段长 prompt 的 KV Cache 在 GPU 1 上, 但下一个相同前缀的请求被路由到 GPU 3, 则缓存命中失败, 仍需重新计算。SGLang 的路由器会尽量将请求发到“已有笔记”的 GPU 上。

### 缓存感知负载均衡的数学建模

可以将分布式 KV Cache 的负载均衡问题建模为带缓存亲和性的请求分配问题:

设有  $M$  个 GPU 节点, 每个节点  $j$  维护一个缓存集合  $\mathcal{C}_j$ 。请求  $i$  的前缀哈希为  $h_i$ , 其“完美分布”为  $j^* = \arg \max_j \text{prefix\_match}(h_i, \mathcal{C}_j)$ 。

负载均衡器的目标是在满足缓存亲和性的条件下最小化节点间负载方差:

$$\min \sum_{j=1}^M (L_j - \bar{L})^2 \quad \text{s.t.} \quad \sum_{i: h_i \in \mathcal{C}_j} w_i \leq L_j$$

其中  $L_j$  为节点  $j$  的负载,  $\bar{L}$  为平均负载,  $w_i$  为请求  $i$  的计算权重。

实际实现中, SGLang 使用一致性哈希的变体, 将前缀映射到候选 GPU 集合, 再从中选择负载最低的节点。

### 嘉宾观点: Radix Tree 的远见 (视频 08:45–10:30)

朱邦华 (RadixArk 联合创始人):

现在的智能体世界里, 很多时候会有很多共享的系统提示词, 共享的系统提示词之后会有不同的用户提示词, 在同一用户提示词下面大家会接着问问题, 所以树状结构不断向下延伸。这也是 SGLang 论文比较有远见的地方——它在 2023 年的时候就已经大概预见到了这样的模式。随着 Agent 编程的用户越来越多, Radix Tree 这套系统带来的增益越来越大。

## 3.4 本章小结

KV Cache 复用是推理优化的第一战场。从单个请求的缓存到跨请求跨机器的共享, SGLang 通过 RadixAttention (基数树组织) + Cache-aware Scheduling (缓存感知调度) + 分布式负载均衡的三层设计, 将重复计算降到最低。在 Agent 工作流场景下, 系统提示词的前缀命中率可接近 100%, 直接消除了业界所称的“推理税”。

## 4 优化“等待”: PD 分离与连续批处理

### 4.1 请求级等待: Continuous Batching

推理引擎的调度方式经历了三代演进:

1. **排队制 (Naive)**: 请求一个一个来, 前一个算完再算下一个。GPU 利用率极低。
2. **批处理 (Static Batching)**: 凑够一批一起算。但凑批需要等待, 且一批中长短请求混在一起, 短请求算完要等长请求才能一起“下车”。

3. **连续批处理 (Continuous Batching)**: 不再等一批人到齐再发车, 而是像一辆随时上下客的公交车——新请求随时上车, 算完的请求随时下车, GPU 始终保持满载。

### Continuous Batching 的性能提升

这一改造能使 GPU 实际吞吐量提升 **2 到 4 倍**。其核心思想是解耦请求的生命周期管理:

- **动态插入**: 新请求到达时, 立即在当前 batch 的迭代边界插入, 无需等待凑批
- **动态驱逐**: 请求生成完 EOS token 后立即从 batch 中移除, 释放 KV Cache 空间
- **迭代粒度**: 每个 forward pass 结束后重新评估 batch 组成, 实现“迭代级”的动态调度

数学上, 吞吐量提升来自 batch size 的持续满载。设 GPU 能同时处理的最大 batch size 为  $B_{\max}$ , 静态批处理的平均 batch size 为  $\bar{B}_{\text{static}} \approx 0.5B_{\max}$  (因为凑批和尾延效应)。连续批处理则可达  $\bar{B}_{\text{CB}} \approx 0.9B_{\max}$  以上, 即吞吐量提升约  $\frac{0.9}{0.5} = 1.8\times$ , 配合 KV Cache 管理的改进可达  $2\sim 4\times$ 。



图 9: 视频中的 Continuous Batching 示意图<sup>7</sup>

视频画面时间区间: 00:16:00。

## 4.2 阶段级等待: Prefill/Decode 分离

大模型推理包含两个性质截然不同的阶段:

- **Prefill (预填充/“读题”阶段)**: 一次性处理用户输入的完整 prompt, 对算力要求高 (计算密集型)。需要并行处理大量 token, 充分利用 GPU 的 Tensor Core 算力。
- **Decode (生成/“答题”阶段)**: 逐 token 生成输出, 每个字都需要翻一遍完整的 KV Cache, 对显存带宽极其敏感 (内存密集型)。计算量不大, 但每次 forward pass 都要访问全部 KV Cache。

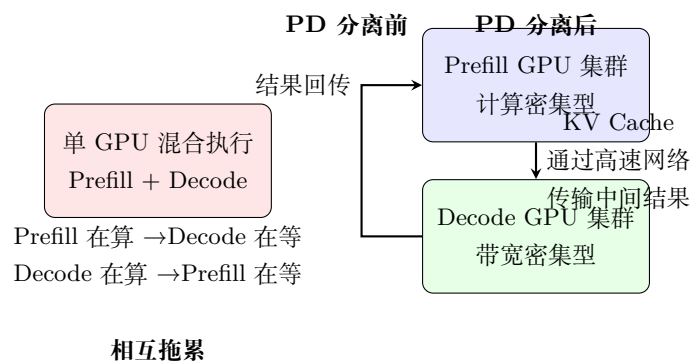


图 10: Prefill/Decode 分离架构：将计算密集的 prefill 与带宽密集的 decode 拆到不同 GPU 集群，各自使用最优硬件配置。

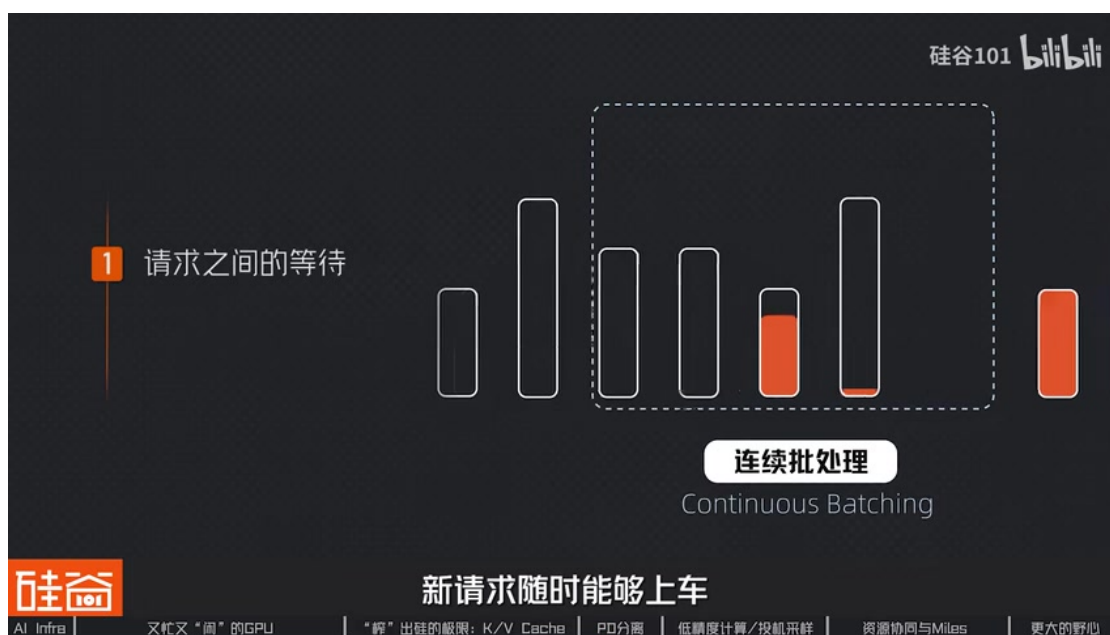


图 11: 视频中的 PD 分离架构示意图<sup>8</sup>

PD 分离后，Prefill GPU 可以用高算力密度的芯片（如 H100/B200），Decode GPU 可以用高带宽的芯片。DeepSeek 就采用了此方案——Prefill 阶段用 32 张 GPU 组成最小计算单元读题，逐字作答时交给另一批 GPU 接手，两拨卡各干各的互不打扰。

### PD 分离的数学分析

设 Prefill 阶段的计算时间为  $T_P$ ，Decode 阶段总时间为  $T_D$ 。在非分离模式下，两个阶段交替执行的总时间为  $T_P + T_D$ （忽略重叠）。

PD 分离后，Prefill 节点处理完立即释放，可投入下一个请求的 Prefill；Decode 节点持续进行逐 token 生成。理想情况下吞吐量提升为：

$$\text{Speedup} = \frac{T_P + T_D}{\max(T_P, T_D/N_D)}$$

其中  $N_D$  为 Decode 节点数量。在典型配置下 ( $N_P : N_D \approx 1 : 3 \sim 1 : 8$ )，可实现接近  $N_D$  倍的 Decode 吞吐量提升。

但 PD 分离的代价是：需要通过网络传输 KV Cache，引入额外延迟  $T_{\text{transfer}}$ 。这个开销取决于网络带宽和 KV Cache 大小。对于 8K 上下文、FP16 精度，单请求 KV Cache 约需传输 20GB 数据，在 InfiniBand NDR400 (400GB/s) 上约需 50ms。当生成 token 数足够多（长输出场景），传输开销被摊薄，PD 分离的收益远大于代价。

### SGLang 的 PD 分离优势

如今 PD 分离已成为推理引擎最重要的架构演进之一。NVIDIA 推出的分布式推理框架 Dynamo 将 PD 分离放在架构设计核心位置，而 **SGLang 是业界最早支持大规模 PD 分离部署的推理引擎之一**。其 KV Cache 传输引擎 (KV Transfer Engine) 经过了大量工程优化，支持多种传输协议和并行策略，能够根据网络拓扑和模型架构自动选择最优配置。

## 4.3 本章小结

等待是 GPU 利用率低下的第二大罪魁祸首。请求级等待通过 Continuous Batching 解决（吞吐量提升 2-4 倍），阶段级等待通过 PD 分离解决（计算密集型与带宽密集型任务各自用最优硬件）。这两项技术已成为现代推理引擎的标准配置，SGLang 在这两个方面都处于业界领先地位。

## 5 优化“算力”：低精度计算与投机采样

### 5.1 GPU 算力为何跑不满

NVIDIA H100 的 Tensor Core 算力相比 A100 提升了数倍，但 AI 推理性能并未同比提升。根本原因有两个：

1. **数据搬运瓶颈**：GPU 大量时间花在从 HBM 搬运数据到计算单元上，计算单元在等待数据
2. **串行解码限制**：自回归生成本质上是串行的——每次只能生成一个 token，无法充分利用 GPU 的并行计算能力

### 5.2 低精度计算：用更少的 bit 算更多

大模型运行时，GPU 显存中主要装载三类数据：**模型权重**、**中间激活值**、以及 **KV Cache**。这三部分的精度可以独立设置。使用更小的数据格式存储或运算任何一部分，都能减少显存占用并利用 GPU 上更快的低精度算力。



## 量化精度与显存-算力权衡

常见的低精度格式及其特点：

- **FP16/BF16**：半精度浮点，模型权重标准格式。与 FP32 相比，显存减半，算力翻倍（H100 上 FP16 Tensor Core 算力为 989 TFLOPS vs FP32 的 67 TFLOPS）
- **INT8**：8 位整数量化。权重和激活值压缩到 8 位，显存再减半。在 H100 上 INT8 算力可达 1979 TOPS
- **FP8**：NVIDIA Hopper 架构原生支持的 8 位浮点。相比 INT8，动态范围更适合神经网络激活值。H100 上 FP8 算力同 INT8，约 1979 TFLOPS
- **INT4/FP4**：4 位量化，显存仅为 FP16 的 1/4。Blackwell 架构（B200）原生支持 FP4，算力进一步提升

量化对精度的影响：设原始权重为  $W$ ，量化后的权重为  $\hat{W} = \text{round}(W/s) \cdot s$ ，其中  $s$  为量化步长。量化误差的均方为：

$$\mathbb{E}[||W - \hat{W}||^2] \approx \frac{1}{12} \cdot s^2 \cdot \dim(W)$$

其中  $\frac{1}{12}$  来自均匀量化假设。选择合适的量化方案（如 LLM.int8() 的混合精度、GPTQ 的最优量化），可在几乎不损失模型质量的前提下实现 2–8 倍的显存和算力提升。



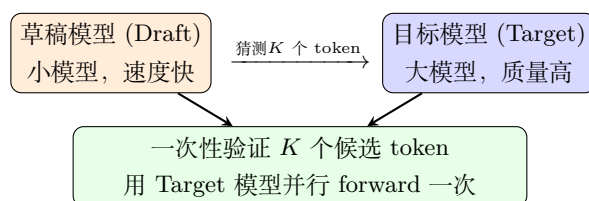
图 12: 视频中的低精度计算示意图<sup>9</sup>

在合适的场景下，通过 FP8 权重量化 + INT8 KV Cache + BF16 激活值的组合，可多释放出一倍的算力空间。

视频画面时间区间：00:17:30。

### 5.3 投机采样 (Speculative Decoding)

投机采样是另一条释放 GPU 算力的路径。其核心思想是：用更快的近似代替精确计算，然后验证。



全部正确 → 接受  $K$  个 token，一次 forward 生成多个  
第  $m$  个错误 → 接受前  $m - 1$  个，从错处重来

图 13: 投机采样流程：草稿模型快速“猜测”  $K$  个 token，目标模型一次性验证，猜对则一次生成多个。



图 14: 视频中的投机采样示意图<sup>10</sup>

### 投机采样的数学增益

设草稿模型的 token 接受率为  $\alpha$ （每次猜测被目标模型接受的概率），每次猜测  $K$  个 token。则期望每次迭代生成的 token 数为：

$$\mathbb{E}[\text{tokens\_per\_iter}] = \frac{1 - \alpha^{K+1}}{1 - \alpha}$$

当  $\alpha = 0.8$ ,  $K = 5$  时，期望生成  $\frac{1-0.8^6}{1-0.8} \approx 3.69$  个 token/迭代，即**加速比约为 3.7 倍**。

最优  $K$  的选择取决于  $\alpha$  和目标模型的验证开销。验证  $K$  个 token 的复杂度为  $O(K)$ （并行 forward），而草稿模型生成  $K$  个 token 为  $O(K)$ （串行但每次开销小）。设草稿模型每次 token 生成开销为目标模型的  $c_d \ll 1$  倍，则总加速比为：

$$\text{Speedup} \approx \frac{1}{c_d + 1/\mathbb{E}[\text{tokens\_per\_iter}]}$$

**最佳情况下，投机采样能将生成速度提升 2 到 3 倍。**

### 嘉宾观点：推理引擎是技术集成枢纽（视频 16:54–19:00）

**朱邦华（RadixArk 联合创始人）：**

SGLang 本身是一个整体的解决方案，它是一个 inference engine。它需要保证 kernel 优化、scheduler 优化、CUDA Graph、Continuous Batching，甚至往上还有 routing、prefix cache 的路由等等。SGLang 有非常非常多的技术组合在一起，把整个推理方案做到极致。像投机采样这样的新能力，推理引擎扮演的角色就是第一时间集成到系统里，把软件和硬件的四层全部打通，让优化协同起来。

一个重要趋势是：**推理引擎与芯片厂商正在形成深度绑定的联合优化**。芯片厂在设计新硬件时就和 SGLang 团队做联合调优，硬件正式发布时 SGLang 已实现 Day-0 支持。SGLang 甚至已成为硬件评测的事实标准——AMD 和 NVIDIA 的新硬件发布当天就会公布在 SGLang 上的性能数据。



## 推理引擎的技术栈全景

SGLang 整合的关键技术模块：

- **Kernel 优化**：手写 CUDA/Triton kernel，针对特定算子（如 FlashAttention、FlashInfer）进行极致优化
- **Scheduler 优化**：Continuous Batching、RadixAttention 缓存感知调度、PD 分离调度
- **CUDA Graph**：将多个 kernel 调用“录制”成可重放的执行图，消除 kernel launch 开销
- **内存管理**：PagedAttention-like 的分页 KV Cache 管理，减少碎片化
- **请求路由**：Cache-aware 负载均衡、分布式前缀路由
- **投机解码**：Draft model 集成、自适应  $K$  值调整
- **多硬件后端**：支持 NVIDIA CUDA、AMD ROCm、Intel Gaudi、Google TPU 等



图 15: 视频中的 SGLang 技术栈示意图<sup>11</sup>

视频画面时间区间：00:19:30。

## 5.4 本章小结

算力释放的两条路径——低精度计算与投机采样——从不同角度榨取 GPU 的潜力。低精度通过减少每个操作的开销来释放算力（可额外释放一倍算力），投机采样通过并行化串行瓶颈来加速（生成速度提升 2-3 倍）。推理引擎将这些技术打包为整体解决方案，并与芯片厂商深度协同，实现 Day-0 的硬件支持。

## 6 资源协同：适配 RL 后训练的 Miles

### 6.1 强化学习对训练-推理协同的新要求

以 OpenAI o1、DeepSeek R1 为代表的新一代推理模型的训练范式发生了根本变化。传统预训练是“体质单一”的任务——喂数据、更新参数、再喂数据，GPU 干的是同一类活。但强化学习需要把推理（生成回答）、评估（打分）、参数更新三种完全不同的计算交替循环执行。

#### RL 训练中的资源浪费

在 RL 训练中，如果用传统训练框架，推理和训练在同一批 GPU 中争抢资源：

- **生成阶段**：模型做推理，产生 batch of responses。此时训练参数更新部分在等待。
- **打分 + 更新阶段**：Reward model 评估 + PPO/GRPO 参数更新。此时推理部分在等待。
- **阶段切换**：每次切换涉及模型权重的同步、计算图的切换、KV Cache 的清除和重建

大量算力被浪费在阶段切换和相互等待上。因此，资源协同成为 RL 后训练效率提升的核心难题。

### 6.2 Miles: SGLang 的训练搭档



图 16: 视频中 Miles RL 训练循环示意图<sup>12</sup>

RadixArk 除了 SGLang 推理引擎外，还推出了训练框架 **Miles**——专门面向大模型后训练 (Post-Training) 的开源训练框架。后训练指模型获得基础能力后，通过 SFT、RL 等方式继续打磨其指令遵循和推理能力。

<sup>12</sup>视频画面时间区间：00:22:00。

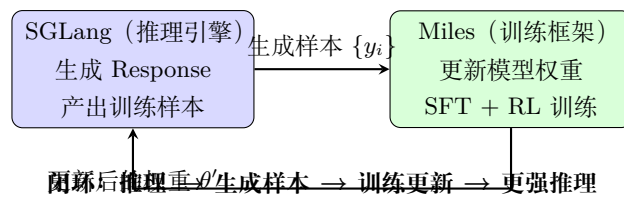


图 17: Miles 与 SGLang 的协同：推理产出样本，训练更新权重，形成高效后训练闭环。



图 18: 视频中 Miles-SGLang 协同架构示意图<sup>13</sup>

## Miles 的关键工程优化

与通用训练框架（如 Megatron-LM、DeepSpeed）相比，Miles 的核心差异在于：

- **训练-推理权重共享**：SGLang 和 Miles 使用相同的模型权重格式和 Kernel 实现，消除推理与训练之间的权重转换开销
- **MoE 路由一致性**：混合专家模型（MoE）的训练和推理路由策略不同，Miles 确保训练时的路由分布与推理时一致，避免“训练-推理漂移”（Training-Inference Mismatch）
- **RL 调度优化**：对 RL 的 rollout-打分-update 循环进行流水线化，让推理和训练在不同的 GPU 子集上并行执行，而非交替等待
- **新硬件 Day-0 支持**：与 SGLang 保持同步，在新 GPU/TPU 发布时同时支持推理和训练

**流水线加速建模**：设 rollout 生成时间  $T_R$ ，打分时间  $T_S$ ，参数更新时间  $T_U$ 。传统串行模式总时间  $T_{\text{serial}} = T_R + T_S + T_U$ 。

Miles 的流水线模式下，将  $N$  个 RL 迭代分布在  $P$  个 GPU 子集上，可实现：

$$T_{\text{pipeline}} \approx \max(T_R, T_S, T_U) + \frac{N-1}{N} \cdot \min(T_R, T_S, T_U)$$

当三者接近均衡时（ $T_R \approx T_S \approx T_U = T$ ），加速比趋近于  $\frac{3T}{\max(T, T, T)} \approx 3\times$ 。

## 嘉宾观点：为什么不只做推理（视频 21:03–23:30）

**陈震林 Richard (RadixArk MTS)：**

如果单纯无视训练、只做推理，反而可能没办法押注未来。最终可能还是会有很多人希望自己训练自己的模型，然后再去推理。如果我们想做整体的 AI 软件基础设施，两边都需要关注。很多人用 SGLang，再用 Miles，本身会带来更强的训练能力，对推理任务、长程任务、还有 Agent 任务提高更多稳定性。MoE 的 routing 在训练和推理中并行策略和最终准确率都会不一样，所以需要更强的工程能力把这件事做成。

## 6.3 本章小结

后训练（特别是 RL 训练）的普及让训练-推理协同成为新的效率瓶颈。Miles 作为 SGLang 的训练搭档，通过权重共享、MoE 路由一致性、RL 流水线调度等工程优化，将后训练流程的效率提升数倍。自 2025 年 11 月启动以来，Miles 已被多家前沿实验室采用，在 DeepSeek-V4 发布当天即实现了全球首个同时支持推理与 RL 训练的开源技术栈。

# 7 更大的野心：AI Infra 成为公共品

## 7.1 开源推理引擎的商业化浪潮

SGLang 和 vLLM 都曾是靠研究者与工程师业余维护的开源项目。当全球面临算力紧缺、需要系统性释放硅的性能时，仅靠开源社区协作已难以满足需求。于是，两个项目的主创团队先后选择创业：

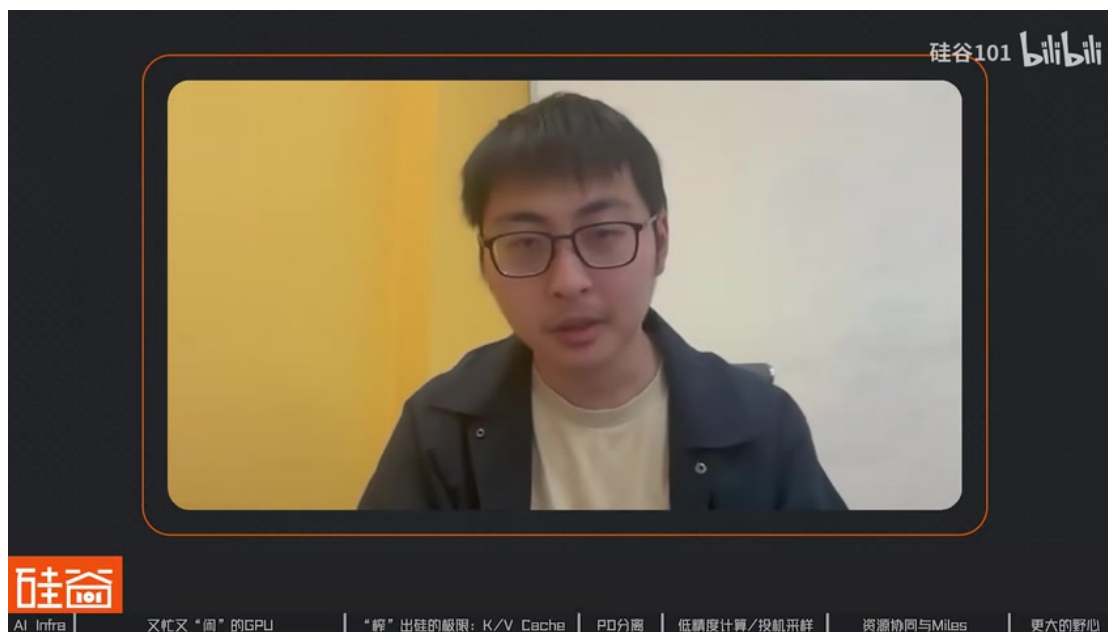


图 19: 视频中 RadixArk 投资人阵容示意图<sup>14</sup>

- **RadixArk** (SGLang 孵化): 种子轮融资超 1 亿美元, 机构投资人包括 NVIDIA、AMD、Intel、联发科、Databricks 等; 个人投资者涵盖 Intel CEO 陈立武、博通 CEO 陈福阳、OpenAI 联合创始人 John Schulman、PyTorch 之父 Soumith Chintala 等
- **vLLM 商业化**: 同样获得超 1 亿美元种子轮融资

这背后的逻辑是: 推理框架正在成为 AI 时代的“操作系统”, 连接模型与芯片的关键层。

#### 战略价值: 为什么三家芯片巨头共同投资

NVIDIA、AMD、Intel 虽互为竞争对手, 却罕见地同时投资 RadixArk, 反映了 AI 产业的深层结构性需求:

- **NVIDIA 的防守与扩张**: 投资 SGLang 能最大化 H100/B200 等 GPU 的利用率, 确保最强开源引擎在自家 GPU 上跑得最顺, 让客户为高昂硬件买单
- **AMD 与 Intel 的破局**: 需要中立、开源且强大的软标准来打破 CUDA 垄断, 让 MI300 系列和 Gaudi 芯片获得公平竞争的平台
- **连接算力与应用的入口**: 推理引擎成为汇聚模型、企业应用和 Agent 到各自算力生态的枢纽

<sup>14</sup> 视频画面时间区间: 00:27:00。

## 7.2 “以后没有 Frontier Lab 这个词”

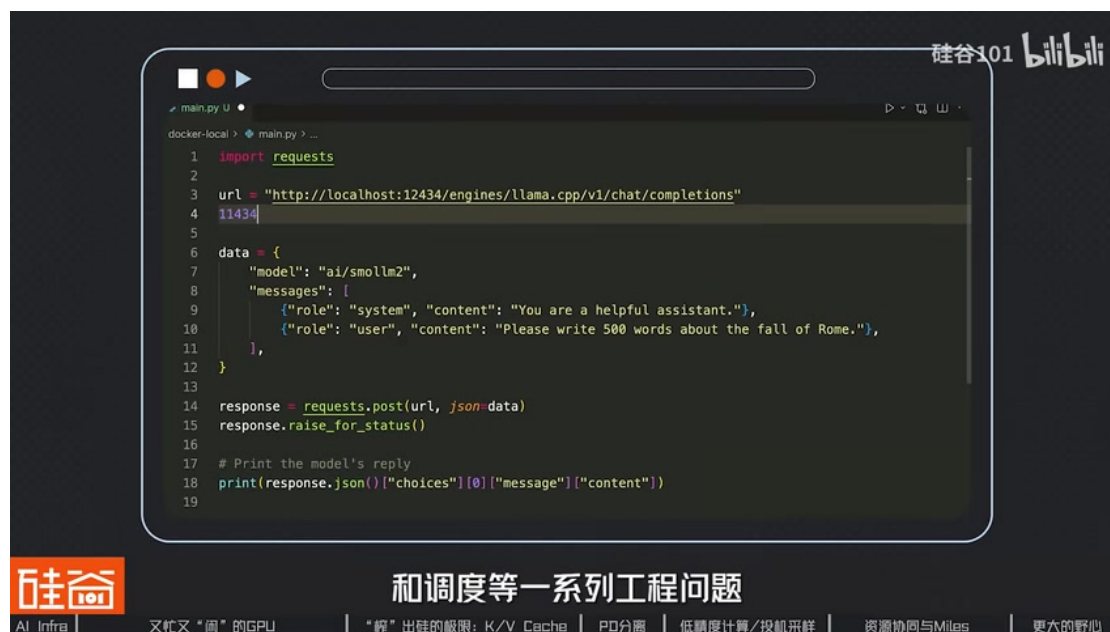


图 20: 视频中 RadixArk 愿景示意图<sup>15</sup>

### RadixArk 的愿景（视频 25:50–28:00）

朱邦华（RadixArk 联合创始人）：

我们做 RadixArk 一个更大的目的，其实是希望以后就不要有 frontier lab（前沿实验室）这个词，因为人人都可以做前沿实验室。前沿本身也是非常动态的事情——两年前大家觉得 Anthropic 不是前沿公司，之前觉得谷歌离 OpenAI 太远追不上。AI 变化太快了。

我们相信会有越来越多的挑战者，有自己想法的人做出更好的 AI 模型和产品。我们做 RadixArk，就是希望能赋能这些更有想法的人，给他们提供 infra 上最强的支持。他们在想做的这一刻开始，就会有和现在的前沿实验室一战之力。最终，可能世界上所有人都能自己训练出前沿级别的模型，做最好的推理。我们更希望是这样一种支持者的角色。

这个愿景的核心是：当 AI Infra 能力从少数巨头的独家资源变成任何创业公司都能直接调用的公共品，AI 行业的准入门槛将被显著降低。

黄仁勋牵头成立的开源 AI 联盟即是这一趋势的体现——联盟中既有模型公司、芯片公司，也有 SGLang 这样的 AI Infra 项目。开源模型迭代速度越来越快（DeepSeek V4、智谱 GLM-5.2、Kimi K3），但开源模型开放的只是权重，权重不会自动变成 Token。部署、显存、吞吐、调度等工程问题仍需解决，而 SGLang 正是连接模型与 Token 生产的关键环节。

<sup>15</sup> 视频画面时间区间：00:26:30。



## AI Infra 的经济学影响

AI Infra 的边际改善能够撬动巨大的经济杠杆：

- 全球 AI 数据中心年投资规模预计 2030 年达 4 万亿美元
- GPU 利用率每提升 1 个百分点，相当于释放出约 400 亿美元年度等效算力
- 推理引擎从 10% 利用率提升到 90% 利用率，等效于将数据中心算力供给提升 9 倍（不增加硬件成本）
- 高性能推理引擎降低了开源模型的使用门槛，加速了 AI 民主化进程

## 7.3 本章小结

RadixArk 的野心超越了单纯的技术优化——他们希望将 AI Infra 能力从少数巨头的壁垒变为公共基础设施。正如云计算改变了服务器行业一样，推理引擎有望成为 AI 时代的操作系统层，让任何开发者都能在模型发布的当天就完成部署，且同一套框架支持 NVIDIA、AMD、TPU 等多种芯片。这一愿景若能实现，AI 行业的创新门槛将被大幅降低，“榨出硅的极限”的意义将远超技术本身。

# 8 总结与延伸

## 8.1 核心脉络回顾

本视频从“GPU 看似繁忙实则大量空转”这一反直觉事实出发，系统梳理了 AI Infra 优化的四大方向：

1. **消除重复计算** (KV Cache 复用)：通过 RadixAttention 的基数树结构，将 KV Cache 从请求级私有缓存变为全局可复用资源，结合缓存感知调度和分布式负载均衡，在 Agent 场景下前缀命中率接近 100%
2. **消除等待** (Continuous Batching + PD 分离)：Continuous Batching 解耦请求生命周期，吞吐量提升 2-4 倍；PD 分离将计算密集型 Prefill 与带宽密集型 Decode 分配到不同 GPU 集群，消除阶段间的相互等待
3. **释放算力** (低精度 + 投机采样)：低精度计算 (FP8/INT8/FP4) 减少显存与算力开销，可额外释放一倍算力空间；投机采样用小模型草稿 + 大模型验证，将串行解码瓶颈并行化，生成速度提升 2-3 倍
4. **协同资源** (Miles + SGLang RL 闭环)：RL 后训练需要推理-评估-更新的交替循环，Miles 通过权重共享、MoE 路由一致性、流水线调度将这一协同效率提升数倍

## 8.2 技术趋势与展望

从 2026 年的 AI Infra 格局来看，几个趋势已经清晰：

- **推理引擎成为基础设施平台**：SGLang 已部署在超 40 万张 GPU 上，支持所有主流模型架构的 Day-0 部署，成为 AI 时代的“操作系统”层

- **软硬件深度联合优化**：芯片设计阶段即与推理引擎团队协同调优，硬件发布时引擎已就绪
- **训练-推理一体化**：Miles 的出现标志着训练和推理框架的边界正在消融，RL 成为二者融合的催化剂
- **AI 民主化**：当 AI Infra 从巨头壁垒变为公共品，“人人都能做前沿实验室”的愿景正在技术层面成为可能

### 8.3 开放问题与延伸阅读

- **KV Cache 压缩**：除复用外，能否通过有损压缩进一步减少 KV Cache 的显存占用？GQA (Grouped Query Attention)、MLA (Multi-head Latent Attention, DeepSeek-V2 提出) 等架构创新正在这个方向上探索
- **芯片级 KV Cache**：谷歌 Frozen v2 等项目试图将部分模型架构固化进芯片，未来可能出现原生支持 KV Cache 管理的专用推理芯片
- **能耗-算力的帕累托前沿**：在 CMU 揭示的能耗浪费背景下，如何量化 AI Infra 优化的能耗-性能权衡，找到全局最优的配置策略

当“榨出硅的极限”从口号变为工程实践，AI 行业的创新重心正在从模型层向基础设施层转移。

这或许是将 AGI 带给更多人的第一步。